

React Tutorial Notes

React Tutorial Notes

Syllabus

Unit I React Basics and JSX: Introduction to React, Advantages of React, Virtual DOM, Creating React App, Understanding JSX, Components (Class vs Functional), Props, Rendering elements, React Developer Tools. 06

Unit II State and Event Handling: useState, Event handling, Conditional rendering, Lists and Keys, Forms in React, Controlled vs Uncontrolled components, useEffect, Lifecycle in Functional components. 06

Unit III Routing and Component Communication: React Router (v6), Route Params, Nested Routes, Navigation, Passing Data via Props, Context API, useContext, Custom Hooks (Intro). 06

Unit IV API Integration and Real-World Features: Fetching data using fetch/axios, Asynchronous programming with Promises, API error handling, Loading states, Pagination, Search, Filters. LocalStorage, Custom Hooks (in detail). 06

Unit V Project Development and Deployment: Building a complete project (e.g., Blog, E-commerce UI), Component structure, Styling in React (CSS Modules / Styled Components), Environment Variables, Deployment on Netlify/Vercel, Code splitting, Performance optimization. 06

UNIT-1

1. Introduction to React

React is a JavaScript library developed by Facebook for building user interfaces (UIs), especially for single-page applications (SPAs). React focuses only on the view layer (UI) of the app — not on routing or backend logic.

React is an open-source, component-based JavaScript library used to build interactive and reusable user interfaces for web applications. It was developed by Facebook in 2013.

React focuses only on the view layer (UI) of the app — not on routing or backend logic.

React allows developers to build components—reusable and isolated chunks of code that return HTML using JSX syntax.

React follows a declarative approach, which means developers describe what they want the UI to look like, and React takes care of updating it.

Key Features / Purpose /Concepts :

To efficiently update and render components when data changes.

- Component-based architecture
- Declarative code style
- Unidirectional data flow
- Maintained by Facebook and a large community
- One-Way Data Binding
- Virtual DOM
- Declarative Programming
- JSX (JavaScript + XML)
- Support for hooks (in functional components)

Example:

```
function Hello() { return <h1>Hello, React!</h1>;}
```

This simple React functional component returns an HTML heading.

Example 1:

```
function Welcome() {  
  return <h1>Hello, React!</h1>;  
}
```

Example 2:

```
const element = <p>This is a paragraph inside JSX</p>;
```

Example 3:

```
const name = "John";  
const greet = <h2>Welcome, {name}!</h2>;
```

2. Advantages of React / Benefit: React provides several benefits compared to traditional JS frameworks like jQuery, or even older frameworks like AngularJS.

Why use React ,is widely used because of several benefits:

- **Component Reusability:** Break UI into independent components.
- **Faster Performance:** Uses a virtual DOM for quick updates.
- **Strong Community:** Rich ecosystem and tons of packages.
- **SEO Friendly:** Can be optimized for search engines.
- **Cross-Platform:** React Native for mobile app development.
- **React Native:** Use the same logic for mobile app development.
- **Faster Rendering:** Virtual DOM ensures only changed parts update.
- **Easy to Learn:** JSX and modular structure reduce complexity.
- **Rich Ecosystem:** Supported by tools like Redux, React Router, etc.

Example of Reusable Component:

```
function Button(props) { return <button>{props.label}</button>;}  
// Reuse in App<Button label="Submit" /><Button label="Cancel" />
```

Example 1: Reusability

```
function Button(props) { return <button>{props.label}</button>;}  
<Button label="Save" /><Button label="Delete" />
```

Example 2: Clean UI Building

```
function Card(props) { return ( <div className="card">    <h3>{props.title}</h3>  
<p>{props.content}</p>    </div> );}
```

Example 3: Component Composition

```
function App() { return ( <> <Header /> <MainContent /> <Footer /> </> );}
```

3. Virtual DOM:

The Virtual DOM is a lightweight copy of the actual DOM.

React:

Creates a virtual DOM in memory

Updates only the changed elements (using diffing algorithm)

Improves performance and makes UI updates faster

Real DOM vs Virtual DOM:

Real DOM	Virtual DOM
Slower updates	Faster updates
Direct manipulation	Memory-based
Refreshes whole tree	Refreshes only changed nodes

3. Virtual DOM:

The Virtual DOM (VDOM) is a lightweight JavaScript object that is a copy of the real DOM.

When a component's state changes:

A new virtual DOM is created

Compared with the previous virtual DOM

Only the changed parts are updated in the real DOM

The Virtual DOM is a programming concept where a virtual representation of the real DOM is kept in memory and synced with the real DOM.

Real DOM updates the entire tree. Virtual DOM updates only the changed node, reducing load on the browser.

Real DOM updates the entire tree. Virtual DOM updates only the changed node, reducing load on the browser.

React uses it to:

Avoid direct DOM manipulation
Improve performance via diffing algorithm

Working Example:

```
const element = <h1>Hello</h1>; // React will compare this with the old virtual DOM and only update differences
```

This process is called reconciliation and makes rendering faster.

Example 1:

```
const element = <h1>Count: 0</h1>;
```

When count changes, React updates just the changed h1.

Example 2: Manual ReactDOM render

```
ReactDOM.render(<App />, document.getElementById("root"));
```

Example 3: Performance Explanation

Old DOM: re-renders full page

Virtual DOM: re-renders changed components only (like just updating one button or label)

4. Creating a React App:

Theory:

create-react-app is a CLI tool to scaffold a new React project without configuration.

It sets up:

Webpack (for bundling)

Babel (to convert JSX)

ESLint (for linting)

Project folders (src, public)

 Command:

```
npx create-react-app my-app  
cd my-app  
npm start
```

To set up a new React project quickly, use Create React App.

Option 1 /Steps:

```
npx create-react-app my-app  
cd my-app  
npm start
```

This sets up:

- A React project structure
- Babel & Webpack configuration
- File structure including App.js, index.js, etc.

Option 2 / steps:

To start a React project, use the Create React App (CRA) tool.

Steps:

```
npx create-react-app my-app  
cd my-app  
npm start
```

This command does:

Sets up file structure (public/, src/)

Adds React libraries

Configures Babel (for JSX), Webpack, ESLint

Files created:

src/App.js: Main App Component

src/index.js: Entry point for rendering

 Example File Structure:

```
/my-app  
/public  
/src  
  App.js  
  index.js
```

Example 1: App.js

```
function App() {  
  return <h1>Hello from App!</h1>;  
}
```

Example 2: index.js

```
import React from 'react';  
import ReactDOM from 'react-dom';  
import App from './App';
```

```
ReactDOM.render(<App />, document.getElementById('root'));
```

5. Understanding JSX:

JSX stands for JavaScript XML. It allows you to write HTML elements inside JavaScript code. JSX gets converted to `React.createElement()` calls by Babel.

Syntax Example:

```
const heading = <h1>Welcome to JSX</h1>;
```

Syntax Rules:

Use `className` instead of `class`

Only one root element (use `<></>` or `<div>`)

Use camelCase for events (e.g., `onClick`)

Dynamic values: `{expression}` inside JSX

JSX = JavaScript + XML

JSX allows us to write HTML inside JavaScript:


```
const element = <h1>Hello, React!</h1>;
```

JSX gets compiled into `React.createElement()` behind the scenes.

You can't return multiple elements without wrapping them:

```
// Wrongreturn <h1></h1><p></p>;
```

```
// Correctreturn ( <> <h1></h1> <p></p> </> );
```

 Example 1:

```
const name = "Ravi";
```

```
const greet = <h1>Hello, {name}</h1>;
```

 Example 2:

```
function UserInfo() {  
  return (  
    <>  
      <h2>User Info</h2>  
      <p>Email: example@gmail.com</p>  
    </>  
  );  
}
```

 Example 3: Dynamic content

```
const isLoggedIn = true;
```

```
return <p>{isLoggedIn ? "Welcome Back!" : "Please Login"}</p>;
```

6. Components (Class vs Functional):

Theory:

Components are functions or classes that return JSX and are the building blocks of React UI.

☐ Functional Components (Preferred)

Simpler syntax

Supports Hooks

No this keyword

Class Components (Legacy)

Uses this.props and lifecycle methods

Still supported, but less recommended.

React apps are built using components. Each component represents a part of the UI.

- Functional Components (Modern and recommended):

```
function Welcome(props) { return <h1>Hello, {props.name}</h1>;}
```

- Class Components (Older way, still used in legacy code):

```
class Welcome extends React.Component { render() { return <h1>Hello,
{this.props.name}</h1>; }}
```

Functional components support hooks like useState, useEffect, etc.

React components are the building blocks of any React app.

Functional Component (Modern):

```
function Greeting(props) { return <h1>Hello, {props.name}</h1>;}
```

Class Component (Legacy):

```
class Greeting extends React.Component { render() { return <h1>Hello,
{this.props.name}</h1>; }}
```

Functional components are preferred now because they support hooks (like useState, useEffect) and are simpler.

Reusable Functional Component:

```
function Student(props) { return <p>Student Name: {props.name}</p>;}
```

Functional Example 1:

```
function Greet(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

□ Class Example 2:

```
class Greet extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

□ Example 3: Multiple functional components

```
function Header() {  
  return <h1>My App</h1>;  
}
```

```
function Footer() {  
  return <footer>Copyright 2025</footer>;  
}
```

7. Props:

Theory:

Props (short for Properties) are used to pass data from parent to child components. They are read-only and make components dynamic.

Props are passed as attributes in JSX.

Props (Properties) are inputs to components.

Props are short for properties, and they allow data to be passed from parent to child components.

Props are:

Passed from parent to child

Immutable (read-only)

Passed like HTML attributes

Used to customize components

Example:

```
function Greet(props) { return <h1>Hello, {props.name}</h1>;}  
// Usage:<Greet name="Alice" />
```

Example:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
  
// In parent component:  
<Welcome name="Amit" />  
<Welcome name="Priya" />  
Each instance gets its own name prop value.
```

Example 1:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
  
<Welcome name="Aarti" />
```

Example 2:

```
function Product(props) {  
  return (  
    <div>  
      <h2>{props.title}</h2>  
      <p>₹ {props.price}</p>  
    </div>  
  );  
}  
  
<Product title="Book" price="250" />
```

Example 3: Passing multiple props

```
<Student name="Ankit" age="20" />
```

8. Rendering Elements:

Theory:

Rendering in React refers to inserting elements/components into the DOM. React does this using ReactDOM.render() in index.js.

React updates the DOM efficiently using the Virtual DOM.

Rendering in React means showing UI elements on the screen.

React uses ReactDOM.render() to render elements into the DOM.

Rendering means displaying content on the screen.

Basic Example:

```
ReactDOM.render(<App />, document.getElementById('root'));
```

Rendering Inside Component:

```
function App() { const user = "John"; return <h1>Welcome, {user}!</h1>;}
```

Dynamic rendering is done using expressions:

```
<h2>{10 + 20}</h2> // Renders 30
```

You use ReactDOM.render() to display elements:

```
ReactDOM.render(<App />, document.getElementById('root'));
```

Inside components:

```
function App() { return ( <div>    <h1>My App</h1>    <p>Welcome!</p>  </div> );}
```

Example 1: Static rendering

```
ReactDOM.render(<h1>Hello</h1>, document.getElementById("root"));
```



Example 2: Render component

```
ReactDOM.render(<App />, document.getElementById("root"));
```

Example 3: Dynamic data rendering

```
const name = "Karan";  
const element = <h1>Hello, {name}</h1>;  
ReactDOM.render(element, document.getElementById("root"));
```

9. React Developer Tools

Theory:

React Developer Tools is a browser extension that lets developers inspect and debug React components.

Features:

With React DevTools, you can:

View component tree

Inspect props and state

Measure performance

Debug issues in real time

Track re-renders

Check performance bottlenecks

View component tree

Highlight component re-renders

Performance monitoring

 Download:

Available for Chrome and Firefox (search “React Developer Tools” in the extensions store)

 How to Use:

Install the extension

Open DevTools (F12)

Go to  Components tab

Available for:

Chrome

Firefox

- Install from Chrome Web Store: React DevTools

Usage:

Open browser DevTools

Go to “ Components” tab

View all components with their props/state

BASIC PROJECT ON UNIT-1 BASE ON SYLLABUS

/* Project: Student Profile App

Goal: To apply all React Basics (JSX, Props, Components, Rendering, DevTools)

Files:

- **.jsApp**
- **StudentCard.js**
- **data.js**
- **index.js**

***/**

// index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

// App.js

```
import React from 'react';
import StudentCard from './StudentCard';
import students from './data';
```

```
function App() {
  return (
    <div style={{ padding: '20px' }}>
      <h1>Student Profile List</h1>
      {students.map((student, index) => (
        <StudentCard key={index} {...student} />
      ))}
    </div>
```

```

    );
  }

export default App;

// StudentCard.js
import React from 'react';

function StudentCard(props) {
  return (
    <div style={{ border: '1px solid #ccc', borderRadius: '8px', padding: '10px', marginBottom: '10px' }}>
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
      <p>Course: {props.course}</p>
      <p>Email: {props.email}</p>
    </div>
  );
}

export default StudentCard;

// data.js
const students = [
  {
    name: 'Amit Sharma',
    age: 20,
    course: 'Computer Engineering',
    email: 'amit@gmail.com'
  },
  {
    name: 'Neha Patil',
    age: 21,
    course: 'Information Technology',
    email: 'neha@gmail.com'
  },
  {
    name: 'Ravi Kumar',
    age: 19,
    course: 'Electronics',
    email: 'ravi@gmail.com'
  }
];

export default students;

```


Output:

Student Profile List

Amit Sharma

Age: 20

Course: Computer Engineering

Email: amit@gmail.com

Neha Patil

Age: 21

Course: Information Technology

Email: neha@gmail.com

Ravi Kumar

Age: 19

Course: Electronics

Email: ravi@gmail.com

Files Included:

1. index.js – Entry point, renders the app.
2. App.js – Main component that maps data.
3. StudentCard.js – Reusable functional component using props.
4. data.js – Dummy student data array.



Concepts Covered:

JSX

Components (Functional)

Props

Rendering Elements

React Developer Tools (inspect props)

Reusability & Structure

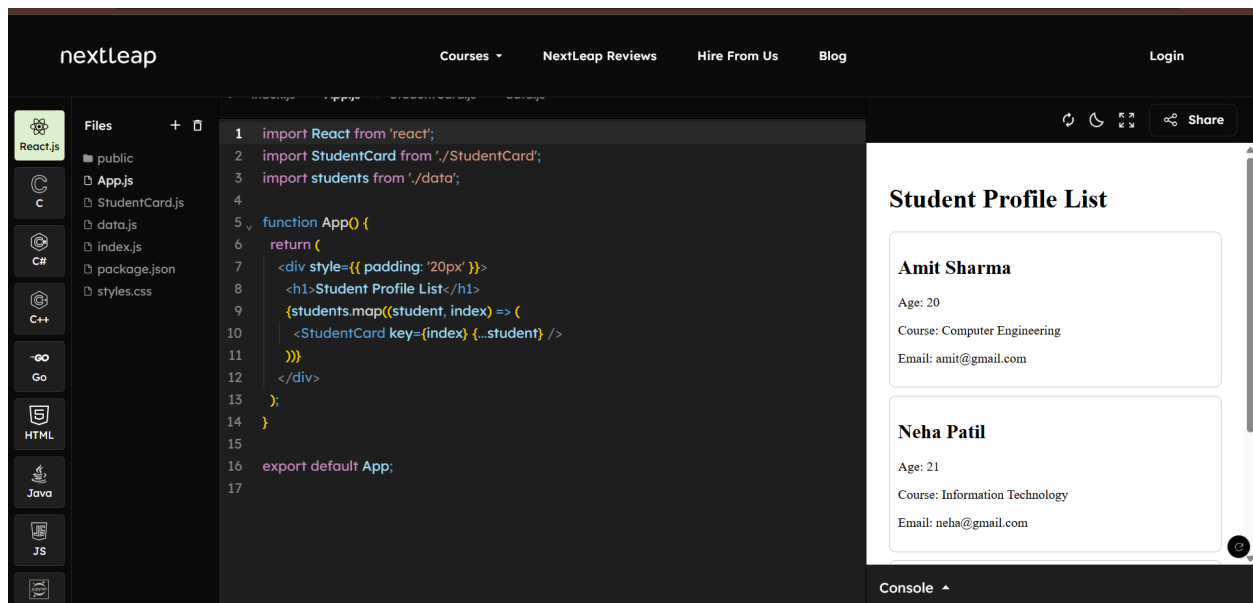
⚙️ No HTML/CSS Setup Needed

Since create-react-app already uses a root index.html, and styling is added via:

```
style={{ padding: '20px' }}
```

you don't need to write any external CSS or HTML manually.

OUTPUT SCREENSHOT



END OF UNIT-1

UNIT 2 NOTES

Syllabus :State and Event Handling: useState, Event handling, Conditional rendering, Lists and Keys, Forms in React, Controlled vs Uncontrolled components, useEffect, Lifecycle in Functional components

Event Handling:

Events are just some actions performed by a user to interact with any application. They can be the smallest of actions, like hovering a mouse pointer on an element that triggers a drop-down menu, resizing an application window, or dragging and dropping elements to upload them etc. Events in React are divided into three categories:

Mouse Events – onClick, onDrag, onDoubleClick

Keyboard Events – onKeyDown, onKeyPress, onKeyUp

Focus Events – onFocus, onBlur

So, every time an event is performed by the user, it usually requires some type of reaction from the application; and these reactions are defined as some functions or blocks of code, called **Event Handlers**. This entire process of working with events using Event Handlers is known as **Event Management**.

Event Management in ReactJS

Event management is one of the important features in a web application. It enables the user to interact with the application. React supports all events available in a web application. React event handling is very similar to DOM events with little changes. Following are some of the common events one can observe in React-based websites –

Clicking on a component.

Scrolling the current page.

Hovering over elements of the current page.

Submitting a form.

Redirecting to another webpage.

Loading images.

Synthetic React Events

In JavaScript, when an event is specified, you will be dealing with a react event type called a **synthetic event** instead of regular DOM events. SyntheticEvent is a simple cross-browser wrapper for native event instances making the events work identically across all browsers. All event handlers must be passed as instances of this wrapper. However, it is expensive in terms of CPU resources as every synthetic event created needs to be garbage-collected. Every synthetic event object has the following attributes:

- boolean bubbles
- boolean cancelable
- DOMEventTarget currentTarget
- boolean defaultPrevented
- number eventPhase
- boolean isTrusted
- DOMEvent nativeEvent
- void preventDefault()
- boolean isDefaultPrevented()
- void stopPropagation()
- boolean isPropagationStopped()
- void persist()
- DOMEventTarget target
- number timeStamp
- string type

Since synthetic events use a lot of resources, they are usually reused and all its properties will be nullified after invoking the event callback to optimize their performance in the browser. SyntheticEvent has the same interface as the native event. And as the synthetic events are authorized by the document node, native events are triggered first followed by the synthetic events.

Adding an Event

As we have already seen, React has the same events as HTML: click, change, mouseover etc. However, the React events are defined with a camelCase and the reaction is written inside the curly braces instead. The syntax of adding an event differs in a functional component and class component.

Following is the syntax to add an onClick event in a **functional component** of React:

```
onClick = {action to be performed}
```

Following is the syntax to add an onClick event in a **class component** of React:

```
onClick = {this.action to be performed}
```

Handling an Event

Let us now learn how to handle these events in a React application with the help of the following step-by-step process.

Define an event handler method to handle the given event.

```
log() {  
  console.log("Event is fired");  
}
```

React provides an alternative syntax using lambda function to define event handler. The lambda syntax is –

```
log = () => {  
  console.log("Event is fired");  
}
```

Regular Method (ES6 class method):

- When you define it like this inside a **class**, it does **not** automatically bind **this**.
- In React **class components**, if you use this method as an event handler, you usually need to do **binding** in the constructor:

```
class App extends React.Component {  
  constructor() {
```

```

    super();
    this.log = this.log.bind(this); // binding required
  }

  log() {
    console.log("Event is fired", this);
  }

  render() {
    return <button onClick={this.log}>Click Me</button>;
  }
}

```

Arrow Function (Class property):

- Arrow functions **do not have their own `this`**.
- Instead, they **inherit `this` from the surrounding context** (lexical `this`).
- In a React class, this means you don't need to bind it in the constructor:

```

class App extends React.Component {
  log = () => {
    console.log("Event is fired", this);
  };

  render() {
    return <button onClick={this.log}>Click Me</button>;
  }
}

```

Key Differences

Regular Method	Arrow Function
----------------	----------------

Needs manual **this** binding in React classes

No need to bind, keeps lexical **this**

Defined on prototype (better for memory)

Defined per instance (uses more memory if many instances)

Good for general methods

Best for event handlers in React classes

✓ Why We Use Arrow Functions for Events in React

- To avoid the headache of **manual binding**
- Makes code shorter & cleaner
- Guarantees **this** will always refer to the component instance

Passing Arguments to Event Handler

There are two methods available to pass arguments to an Event Handler:

Arrow Method

Bind Method

Arrow Method

If you want to know the target of the event, then add an argument **e** in the handler method. React will send the event target details to the handler method.

If you want to know the target of the event, then add an argument **e** in the handler method. React will send the event target details to the handler method.

```
log(e) {  
  console.log("Event is fired");  
  console.log(e.target);  
}
```

```
}
```

The alternative lambda syntax is –

```
log = (e) => {  
  console.log("Event is fired");  
  console.log(e.target);  
}
```

If you want to send extra details during an event, then add the extra details as initial argument and then add argument (**e**) for event target.

```
log(extra, e) {  
  console.log("Event is fired");  
  console.log(e.target);  
  console.log(extra);  
  console.log(this);  
}
```

The alternative lambda syntax is as follows –

```
log = (extra, e) => {  
  console.log("Event is fired");  
  console.log(e.target);  
  console.log(extra);  
  console.log(this);  
}
```

stop

Bind Method

We can also bind the event handler method in the constructor of the component. This will ensure the availability of this in the event handler method.

```
constructor(props) {  
  super(props);  
  this.logContent = this.logContent.bind(this);  
}
```


If the event handler is defined in alternate lambda syntax, then the binding is not needed. this keyword will be automatically bound to the event handler method.

Set the event handler method for the specific event as specified below –

```
<div onClick={this.log}> ... </div>
```

To set extra arguments, bind the event handler method and then pass the extra information as second argument.

```
<div onClick={this.log.bind(this, extra)}> ... </div>
```

The alternate lambda syntax is as follows –

```
<div onClick={this.log(extra, e)}> ... </div>
```

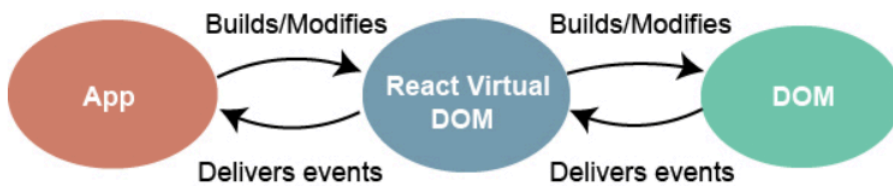
Key Differences (Table)

Feature	Arrow Method (log = () => {})	Bind Method (this.log.bind(this))
this handling	Auto-binds (lexical this)	Needs manual binding
Where it is stored	Per instance (uses more memory)	Prototype (memory efficient)
Ease of use	Very simple, less code	Extra boilerplate (constructor bind)
Common in React today	✅ Yes (preferred)	⚠️ Older React pattern

◆ Summary for Students

- **Arrow Method** → easier, no binding, modern React style.
- **Bind Method** → older pattern, more boilerplate, but slightly more memory efficient.

Events Handler



Handling events with react have some syntactic differences from handling events on DOM. These are:

1. React events are named as **camelCase** instead of **lowercase**.
2. With JSX, a function is passed as the **event handler** instead of a **string**. For example:

EVENT HANDLING CODE EXAMPLE:

```

import React, { Component } from 'react';
class App extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      companyName: ""
    };
  }
  changeText(event) {
    this.setState({
      companyName: event.target.value
    });
  }
  render() {
    return (
      <div>

```

```
    <h2>Simple Event Example</h2>
    <label htmlFor="name">Enter company name: </label>
    <input type="text" id="companyName"
onChange={this.changeText.bind(this)} />
    <h4>You entered: { this.state.companyName } </h4>
  </div>
);
}
}
export default App;
```

Output

When you execute the above code, you will get the following output.



Simple Event Example

Enter company name:

You entered:

After entering the name in the textbox, you will get the output as like below screen.



Simple Event Example

Enter company name:

You entered: www.javatpoint.com

State Management / State handling

The state is an updatable structure that is used to contain data or information about the component. The state in a component can change over time. The change in state over time can happen as a response to user action or system event. A component with the state is known as stateful components. It is the heart of the react component which determines the behavior of the component and how it will render. They are also responsible for making a component dynamic and interactive

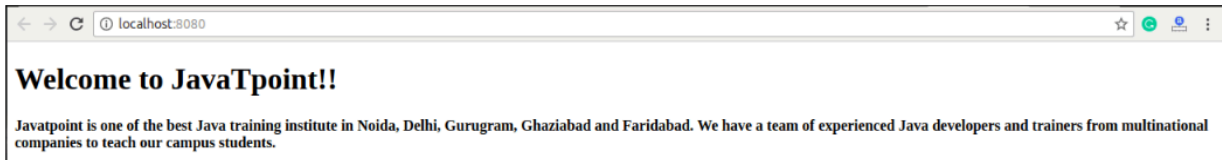
Defining State

To define a state, you have to first declare a default set of values for defining the component's initial state. To do this, add a class constructor which assigns an initial state using `this.state`. The `'this.state'` property can be rendered inside `render()` method.

```
import React, { Component } from 'react';
class App extends React.Component {
  constructor() {
    super();
    this.state = { displayBio: true };
  }
  render() {
    const bio = this.state.displayBio ? (
      <div>
        <p><h3>Javatpoint is one of the best Java training institute in Noida, Delhi,
        Gurugram, Ghaziabad and Faridabad. We have a team of experienced Java developers
        and trainers from multinational companies to teach our campus students.</h3></p>
      </div>
    ) : null;
    return (
      <div>
        <h1> Welcome to JavaTpoint!! </h1>
        { bio }
      </div>
    );
  }
}
```

export default App;

Output



Changing the State:

We can change the component state by using the `setState()` method and passing a new state object as the argument. Now, create a new method `toggleDisplayBio()` in the above example and bind this keyword to the `toggleDisplayBio()` method otherwise we can't access this inside `toggleDisplayBio()` method.

```
this.toggleDisplayBio = this.toggleDisplayBio.bind(this);
```

```
import React, { Component } from 'react';
class App extends React.Component {
  constructor() {
    super();
    this.state = { displayBio: false };
    console.log('Component this', this);
    this.toggleDisplayBio = this.toggleDisplayBio.bind(this);
  }
  toggleDisplayBio() {
    this.setState({ displayBio: !this.state.displayBio });
  }
  render() {
    return (
      <div>
        <h1>Welcome to JavaTpoint!!</h1>
        {
          this.state.displayBio ? (
            <div>
              <p><h4>Javatpoint is one of the best Java training institute in Noida,
                Delhi, Gurugram, Ghaziabad and Faridabad. We have a team of experienced Java
                developers and trainers from multinational companies to teach our campus
                students.</h4></p>
            </div>
          ) : null
        }
      </div>
    );
  }
}
```

```

        <button onClick={this.toggleDisplayBio}> Show Less </button>
      </div>
    ) : (
      <div>
        <button onClick={this.toggleDisplayBio}> Read More </button>
      </div>
    )
  }
</div>
)
}
export default App;

```

Output:



When you click the **Read More** button, you will get the below output, and when you click the **Show Less** button, you will get the output as shown in the above image.



Difference between State and Props

SN	Props	State
1.	Props are read-only.	State changes can be asynchronous.
2.	Props are immutable.	State is mutable.

3.	Props allow you to pass data from one component to other components as an argument.	State holds information about the components.
4.	Props can be accessed by the child component.	State cannot be accessed by child components.
5.	Props are used to communicate between components.	States can be used for rendering dynamic changes with the component.
6.	Stateless component can have Props.	Stateless components cannot have State.
7.	Props make components reusable.	State cannot make components reusable.
8.	Props are external and controlled by whatever renders the component.	The State is internal and controlled by the React Component itself.

What is State?

- **State** is an object that stores **dynamic data** in a component.
- Unlike props (which are passed from parent), state is **managed within the component** itself.
- When state changes, React automatically **re-renders the component** to update the UI.

👉 Think of **state as a component's memory**.

State represents the value of a dynamic properties of a React component at a given instance. React provides a dynamic data store for each component. The internal data represents the state of a React component and can be accessed using this.state member variable of the component. Whenever the state of the component is changed, the component will re-render itself by calling the render() method along with the new state.

As the state of the clock component changes every second, the clock's render() method will be called every second and the render() method show the current time using it's current state.

The simple representation of the state is as follows –

```
{  
  date: '2020-10-10 10:10:10'  
}
```

Why do we need State?

- To keep track of user interactions and changing data.
- Example use cases:
 - Button clicks (counters, likes 👍)
 - Form input values
 - Showing/hiding elements
 - API response storage
 - Toggling light/dark mode

Components in React:

- **Components are the building blocks of a React application.**
- They are **independent, reusable pieces of UI** that define how a part of the interface should look and behave.

- A React app is usually made up of **many small components** combined together.

👉 Think of components like **LEGO blocks** – each block does something small, and when combined, they form a complete structure.

Types of Components in React:

1. Functional Components (Preferred in modern React)

- Simple JavaScript functions.
- Can use **React Hooks** (`useState`, `useEffect`) for state & lifecycle.

👉 Example:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

2. Class Components (Older way)

- ES6 classes that extend `React.Component`.
- Use `this.state` and `this.setState()` for state.

👉 Example:

```
import React, { Component } from "react";  
  
class Welcome extends Component {  
  render() {  
    return <h1>Hello, {this.props.name}!</h1>;  
  }  
}
```

Component Composition (Using Components inside others)

You can build larger components by **combining smaller ones**.

👉 Example:

```
function Header() {  
  return <h2>My Website</h2>;  
}  
  
function App() {  
  return (  
    <div>  
      <Header />  
      <p>Welcome to my website.</p>  
    </div>  
  );  
}
```

State in Functional Components → **useState** Hook

Syntax:

```
const [state, setState] = useState(initialValue);
```

- **state** → current state value
- **setState** → function to update state
- **initialValue** → default value (number, string, boolean, array, object...)

Defining a State

State in React can be used with functional and class components. To work with state in a component, there must exist a starting point, i.e. initial state. This initial state of a component

must be defined in the constructor of the component's class. Following is the syntax to define a state of any Class –

```
state = {attribute: "value"};
```

Let us look at a sample code for a class component with an initial state –

```
Class SampleClass extends React.Component
{
  constructor(props)
  {
    super(props);
    this.state = { name : "John Doe" };
  }
}
```

Creating a state Object

React components have a built-in state object. The state object is used to store all the property values that belong to the component in which this state is defined. When the state object changes, the component re-renders.

Let us look at a sample code to demonstrate how to create a state object in React.

```
Class BookClass extends React.Component
{
  constructor(props)
  {
    super(props);
    this.state = { name : "John Doe" };
  }
  render() {
    return (
      <div>
        <h1>Name of the Author</h1>
      </div>
    );
  }
}
```

}

React State Management APIs

1 `useState` (Local State)

- Basic hook for component-level state.
- Good for **simple, isolated data**.

```
const [count, setCount] = useState(0);
```

2 `useReducer` (Complex Local State)

- Useful when state has **complex logic** (objects, arrays, multiple updates).
- Alternative to `useState`.

```
function reducer(state, action) {  
  switch(action.type) {  
    case "increment": return { count: state.count + 1 };  
    case "decrement": return { count: state.count - 1 };  
    default: return state;  
  }  
}
```

```
const [state, dispatch] = useReducer(reducer, { count: 0 });
```

3 `Context API` (Global State)

- Built-in React API for **prop drilling problems** (passing data deep without props).

- Example: Sharing theme or user info across app.

```
const ThemeContext = React.createContext();

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  return <ThemedButton />;
}

function ThemedButton() {
  const theme = React.useContext(ThemeContext);
  return <button>{theme} mode</button>;
}
```

4 Third-Party State Management Libraries (For Large Apps)

✓ Redux

- Most popular state management library.
- Stores global state in a **single store**.
- Uses actions, reducers, and dispatch.

✓ Zustand

- Lightweight alternative to Redux.
- Uses hooks for state management.

✓ MobX

- Reactive state management.
- Automatically updates UI when data changes.

✓ Recoil

- Experimental library by Facebook for state management.
- Simpler than Redux, integrates well with React.

Lifecycle and State Management APIs

React provides APIs to manage state changes at different **stages of component lifecycle**:

- `useEffect` → sync state with external systems (APIs, localStorage).
- `useContext` → share state globally.
- `useReducer` → advanced local state.

stateful and stateless compount

1. Stateful Component

◆ Concept

- A **stateful component** is a component that **manages its own state** (data that can change).
- It can **store, update, and pass state** to other components.
- Usually written with `useState`, `useReducer`, or **class components**.

👉 Think of it as a **smart / container component**.

◆ Example: Stateful Counter

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0); // state

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}

export default Counter;
```

✓ Here, `Counter` manages its own state → **Stateful Component**.

◆ 2. Stateless Component

◆ Concept

- A **stateless component** is a component that **does not manage any state**.
- It only receives **props** (data from parent) and displays them.
- They are **presentational / UI components**.

👉 Think of it as a **dumb / presentational component**.

◆ Example: Stateless Display

```
function Display({ message }) {
  return <h2>{message}</h2>;
}

export default Display;
```

✓ Here, `Display` just receives `message` via props and renders it → **Stateless Component**.

Difference Table

Feature	Stateful Component	Stateless Component
State	Maintains internal state	No state (only props)
Purpose	Logic + Data Management	UI Presentation
Hooks/Class	Uses <code>useState</code> , <code>useReducer</code> , or class state	Functional (no state hooks)
Reusability	Less reusable (depends on logic)	More reusable (pure UI)
Examples	Forms, Counters, Todo App	Buttons, Display Components, Headers

Summary:

- **Stateful Component** → has state, dynamic, “smart”.
- **Stateless Component** → no state, static, “dumb”, only props.

State Management using React Hooks

What is State Management?

- **State Management** in React means **tracking data changes** inside components and updating the UI when that data changes.
 - Earlier, React class components used `this.state` + `setState()`.
 - Now, with **React Hooks**, we manage state in **functional components** using `useState`, `useReducer`, and `useContext`.
-

◆ Common Hooks for State Management

1. **useState** Hook

- Basic and most commonly used hook for managing local component state.
- Returns a **state variable** and a **setter function**.

👉 Syntax:

```
const [state, setState] = useState(initialValue);
```

👉 Example: Counter

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

✅ Every time **setCount** is called, React re-renders the component with updated state.

2. **useReducer** Hook

- Used when state logic is **complex** (multiple values, actions, conditions).
- Similar to Redux but built into React.

👉 Syntax:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

👉 Example: Counter with Reducer

```
import React, { useReducer } from "react";
```

```
function reducer(state, action) {  
  switch (action.type) {  
    case "increment":  
      return { count: state.count + 1 };  
    case "decrement":  
      return { count: state.count - 1 };  
    case "reset":  
      return { count: 0 };  
    default:  
      return state;  
  }  
}
```

```
function CounterReducer() {  
  const [state, dispatch] = useReducer(reducer, { count: 0 });  
  
  return (  
    <div>  
      <h2>Count: {state.count}</h2>  
      <button onClick={() => dispatch({ type: "increment" })}>+</button>  
      <button onClick={() => dispatch({ type: "decrement" })}>-</button>  
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>  
    </div>  
  );  
}
```

✅ `useReducer` is better for complex state logic (like forms, multiple counters, shopping carts).

3. `useContext` Hook (for Global State)

- Used for **sharing state across multiple components** without prop drilling.
- Often combined with `useReducer` for global state management.

👉 Example: Theme Context

```
import React, { useState, useContext, createContext } from "react";
```

```
const ThemeContext = createContext();
```

```
function ThemeProvider({ children }) {  
  const [dark, setDark] = useState(false);  
  return (  
    <ThemeContext.Provider value={{ dark, setDark }}>  
      {children}  
    </ThemeContext.Provider>  
  );  
}
```

```
function ThemeButton() {  
  const { dark, setDark } = useContext(ThemeContext);  
  return (  
    <button onClick={() => setDark(!dark)}>  
      {dark ? "Switch to Light Mode" : "Switch to Dark Mode"}  
    </button>  
  );  
}
```

```
export default function App() {  
  return (  
    <ThemeProvider>  
      <ThemeButton />  
    </ThemeProvider>  
  );  
}
```

✅ `useContext` avoids passing props through multiple levels of components.

◆ Comparison of Hooks in State Management

Hook	Use Case	Example
<code>useState</code>	Simple, local state	Counter, toggle button
<code>useReducer</code>	Complex state logic, multiple actions	Forms, shopping cart

Component Lifecycle using React Hooks

What is a Component Lifecycle? / functional component lifecycle.

- The **lifecycle of a React component** refers to the different stages it goes through:

Mounting → Component is created and added to the DOM.

Updating → Component re-renders when props or state change.

Unmounting → Component is removed from the DOM.

Mounting example

```
import React, { useEffect } from "react";

function Welcome() {

  useEffect(() => {

    console.log("Component Mounted "); // runs once

  }, []);

  return <h2>Welcome Component</h2>;

}
```

Updating (componentDidUpdate equivalent)

Run effect when specific state/props change → use **[dependencies]**.

```
import React, { useState, useEffect } from "react";

function Counter() {

  const [count, setCount] = useState(0);

  useEffect(() => {

    console.log("Count updated:", count);

  }, [count]); // runs every time "count" changes

  return (

    <div>

      <h2>Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>Increment</button>

    </div>

  );

}
```

Unmounting (componentWillUnmount equivalent)

Use **cleanup function** in **useEffect**.

```
import React, { useState, useEffect } from "react";

function Timer() {

  useEffect(() => {

    const interval = setInterval(() => {
```

```

        console.log("Timer running...");
    }, 1000);

    // Cleanup runs when component unmounts
    return () => {
        clearInterval(interval);

        console.log("Component Unmounted ❌");
    };
}, []);

return <h2>Timer Running...</h2>;
}

export default function App() {
    const [show, setShow] = useState(true);

    return (
        <div>

            <button onClick={() => setShow(!show)}>

                {show ? "Stop Timer" : "Start Timer"}

            </button>

            {show && <Timer />}

        </div>
    );
}

```

```
);  
}
```

- In **class components**, lifecycle methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` are used.
- In **functional components**, we use **React Hooks** (`useEffect`, `useLayoutEffect`, `useRef`) to manage lifecycle.

Mapping – Class vs Functional Lifecycle

Class Component Lifecycle

Functional Component Equivalent

`componentDidMount`

`useEffect(() => {...}, [])`

`componentDidUpdate`

`useEffect(() => {...}, [deps])`

`componentWillUnmount`

`useEffect(() => { return () => {...}; }, [])`

♦ 2. Lifecycle Stages with Hooks

✓ **Mounting (when the component loads first time)**

- Equivalent to `componentDidMount()` in class components.
- Achieved with `useEffect(() => { ... }, [])` (empty dependency array).

👉 Example:

```
import React, { useEffect } from "react";
```

```
function Welcome() {  
  
  useEffect(() => {  
  
    console.log("Component Mounted ✅");  
  
  }, []); // runs only once on mount  
  
  return <h2>Welcome Component</h2>;  
  
}
```

✅ Updating (when state or props change)

- Equivalent to `componentDidUpdate()`.
- Achieved with `useEffect(() => { ... }, [dependency])`.
- Runs whenever the **dependency changes**.

👉 Example:

```
import React, { useState, useEffect } from "react";
```

```
function Counter() {
```



```
const [count, setCount] = useState(0);

useEffect(() => {

  console.log("Count Updated:", count);

}, [count]); // runs when "count" changes

return (

  <div>

    <h2>Count: {count}</h2>

    <button onClick={() => setCount(count + 1)}>Increment</button>

  </div>

);

}
```

✓ Unmounting (when the component is removed from DOM)

- Equivalent to `componentWillUnmount()`.
- Achieved with **cleanup function** inside `useEffect`.

👉 Example:

```
import React, { useState, useEffect } from "react";
```

```
function Timer() {
```

```
useEffect(() => {  
  
  const interval = setInterval(() => {  
  
    console.log("Timer running...");  
  
  }, 1000);  
  
  return () => {  
  
    clearInterval(interval);  
  
    console.log("Component Unmounted ✖, Timer cleared");  
  
  };  
}, []);  
  
return <h2>Timer Running</h2>;  
}
```

```
export default function App() {  
  
  const [show, setShow] = useState(true);  
  
  return (  
  
    <div>  
  
      <button onClick={() => setShow(!show)}>  
  
        {show ? "Stop Timer" : "Start Timer"}  
  
      </button>  
  
      {show && <Timer />}  
  
    </div>  
  
  )  
}
```

```
);  
}
```

◆ 3. `useLayoutEffect` vs `useEffect`

- `useEffect` → Runs **after** render is painted on the screen (non-blocking).
- `useLayoutEffect` → Runs **before** the browser paints (useful for measuring DOM nodes).

👉 Example:

```
useLayoutEffect(() => {  
  console.log("Runs before paint");  
});
```

◆ 4. Lifecycle Mapping (Class vs Hooks)

Class Component Method

Functional Component (Hooks)

`componentDidMount`

`useEffect(() => {...}, [])`

`componentDidUpdate`

`useEffect(() => {...}, [deps])`

`componentWillUnmount`

`useEffect(() => { return () => {...}; }, [])`

Layout in Component

One of the advanced features of React is that it allows arbitrary user interface (UI) content to be passed into the component using properties. As compared to React's special children property, which allows only a single user interface content to be passed into the component, this option enables multiple UI content to be passed into the component. This option can be seen as an extension of children property. One of the use cases of this option is to layout the user interface of a component.

For example, a component with customizable header and footer can use this option to get the custom header and footer through properties and layout the content.

```
ReactDOM.render(  
  <React.StrictMode>  
    <ExpenseEntryItemList items={items}  
      header={  
        <div><h1>Expense manager</h1></div>  
      }  
      footer={  
        <div style={{ textAlign: "left" }}>  
          <p style={{ fontSize: 12 }}>Sample application</p>  
        </div>  
      }  
    </>  
  </React.StrictMode>,  
  document.getElementById('root')  
)
```

Pagination

React provides pagination component through third party UI component library. React community provides a large collection of UI / UX components and it is tough to choose the right library for our requirement. Bootstrap UI library is one of the popular choice for the developer and it is extensively used.

Pagination component

Pagination component allows the developer to create simple pagination with bootstrap design in a web application. Pagination component accepts below components.

- Pagination.Item
- Pagination.First
- Pagination.Last
- Pagination.Previous
- Pagination.Next

Applying Pagination component

First of all, create a new react application and start it using below command.

```
create-react-app myapp  
cd myapp  
npm start
```

Next, install the bootstrap library using below command,

```
npm install --save bootstrap react-bootstrap
```

Next, open App.css (src/App.css) and remove all the CSS classes.

```
// remove the css
```

Conditional Rendering

In React, we can create multiple components which encapsulate behavior that we need. After that, we can render them depending on some conditions or the state of our application. In other words, based on one or several conditions, a component decides which elements it will return. In React, conditional rendering works the same way as the conditions work in JavaScript. We use JavaScript operators to create elements representing the current state, and then React Component update the UI to match them.

From the given scenario, we can understand how conditional rendering works. Consider an example of handling a login/logout button. The login and logout buttons will be separate components. If a user logged in, render the logout component to display the logout button. If a user not logged in, render the login component to display the login button. In React, this situation is called as conditional rendering.

- if
- ternary operator
- logical && operator
- switch case operator
- Conditional Rendering with enums
- ```
function UserLoggin(props) {
```
- ```
  return <h1>Welcome back!</h1>;
```
- ```
}
```
- ```
function GuestLoggin(props) {
```
- ```
 return <h1>Please sign up.</h1>;
```
- ```
}
```
- ```
function SignUp(props) {
```
- ```
  const isLoggedIn = props.isLoggedIn;
```
- ```
 if (isLoggedIn) {
```
- ```
    return <UserLogin />;
```
- ```
 }
```
- ```
  return <GuestLogin />;
```
- ```
}
```
- ```
ReactDOM.render(
```
- ```
 <SignUp isLoggedIn={false} />,
```

- document.getElementById('root')
- );

## Logical && operator

This operator is used for checking the condition. If the condition is true, it will return the element right after &&, and if it is false, React will ignore and skip it.

### Syntax

```
{
 condition &&
 // whatever written after && will be a part of output.
}
```

We can understand the behavior of this concept from the below example.

If you run the below code, you will not see the alert message because the condition is not matching.

```
('javatpoint' == 'JavaTpoint') && alert('This alert will never be shown!')
```

If you run the below code, you will see the alert message because the condition is matching.

```
(10 > 5) && alert('This alert will be shown!')
```

### Example

```
import React from 'react';
import ReactDOM from 'react-dom';
// Example Component
function Example()
{
 return(<div>
 {
 (10 > 5) && alert('This alert will be shown!')
 }
 </div>
);
}
```

You can see in the above output that as the condition (10 > 5) evaluates to true, the alert message is successfully rendered on the screen.

## Ternary operator

The ternary operator is used in cases where two blocks alternate given a certain condition. This operator makes your if-else statement more concise. It takes three operands and is used as a shortcut for the if statement.

### Syntax

condition ? **true** : **false**

If the condition is true, statement1 will be rendered. Otherwise, false will be rendered.

### Example

```
render() {
 const isLoggedIn = this.state.isLoggedIn;
 return (
 <div>
 Welcome {isLoggedIn ? 'Back' : 'Please login first'}.
 </div>
);
}
```

## Switch case operator

Sometimes it is possible to have multiple conditional renderings. In the switch case, conditional rendering is applied based on a different state.

### Example

```
function NotificationMsg({ text }) {
 switch(text) {
 case 'Hi All':
 return <Message: text={text} />;
 case 'Hello JavaTpoint':
 return <Message text={text} />;
 default:
 return null;
 }
}
```

## Conditional Rendering with enums



An enum is a great way to have a multiple conditional rendering. It is more readable as compared to switch case operator. It is perfect for mapping between different state. It is also perfect for mapping in more than one condition. It can be understood in the below example.

## Example

```
function NotificationMsg({ text, state }) {
 return (
 <div>
 {{
 info: <Message text={text} />,
 warning: <Message text={text} />,
 }}[state]
 </div>
);
}
```

## Conditional Rendering Example

In the below example, we have created a stateful component called App which maintains the login control. Here, we create three components representing Logout, Login, and Message component. The stateful component App will render either or depending on its current state.

```
import React, { Component } from 'react';
// Message Component
function Message(props)
{
 if (props.isLoggedIn)
 return <h1>Welcome Back!!!</h1>;
 else
 return <h1>Please Login First!!!</h1>;
}
// Login Component
function Login(props)
{
 return(
 <button onClick = {props.clickInfo}> Login </button>
);
}
```

*// Logout Component*

```
function Logout(props)
{
 return(
 <button onClick = {props.clickInfo}> Logout </button>
);
}
class App extends Component{
 constructor(props)
 {
 super(props);
 this.handleLogin = this.handleLogin.bind(this);
 this.handleLogout = this.handleLogout.bind(this);
 this.state = {isLoggedIn : false};
 }
 handleLogin()
 {
 this.setState({isLoggedIn : true});
 }
 handleLogout()
 {
 this.setState({isLoggedIn : false});
 }
 render(){
 return(
 <div>
 <h1> Conditional Rendering Example </h1>
 <Message isLoggedIn = {this.state.isLoggedIn}/>
 {
 (this.state.isLoggedIn)?(
 <Logout clickInfo = {this.handleLogout} />
) : (
 <Login clickInfo = {this.handleLogin} />
)
 }
 </div>
);
 }
}
export default App;
```

### Output:

When you execute the above code, you will get the following screen.



After clicking the logout button, you will get the below screen.



## Lists and Keys

Lists are used to display data in an ordered format and mainly used to display menus on websites. In React, Lists can be created in a similar way as we create lists in JavaScript. Let us see how we transform Lists in regular JavaScript.

The `map()` function is used for traversing the lists. In the below example, the `map()` function takes an array of numbers and multiply their values with 5. We assign the new array returned by `map()` to the variable `multiplyNums` and log it.

### Example

```
var numbers = [1, 2, 3, 4, 5];
const multiplyNums = numbers.map((number)=>{
 return (number * 5);
});
console.log(multiplyNums);
```

### Output

The above JavaScript code will log the output on the console. The output of the code is given below.

*[5, 10, 15, 20, 25]*

```
import React from 'react';
import ReactDOM from 'react-dom';

const myList = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];
const listItems = myList.map((myList)=>{
 return {myList};
});
ReactDOM.render(
 {listItems} ,
 document.getElementById('app')
);
export default App;
```

Output



## Rendering Lists inside components

In the previous example, we had directly rendered the list to the DOM. But it is not a good practice to render lists in React. In React, we had already seen that everything is built as individual components. Hence, we would need to render lists inside a component. We can understand it in the following code.

Example

```
import React from 'react';
import ReactDOM from 'react-dom';

function NameList(props) {
 const myLists = props.myLists;
```

```

const listItems = myLists.map((myList) =>
 {myList}
);
return (
 <div>
 <h2>Rendering Lists inside component</h2>
 {listItems}
 </div>
);
}
const myLists = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];
ReactDOM.render(
 <NameList myLists={myLists} />,
 document.getElementById('app')
);
export default App;

```

## Output



## Keys

A key is a unique identifier. In React, it is used to identify which items have changed, updated, or deleted from the Lists. It is useful when we dynamically created components or when the users alter the lists. It also helps to determine which components in a collection needs to be re-rendered instead of re-rendering the entire set of components every time.

```

const stringLists = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];

const updatedLists = stringLists.map((strList)=>{
 <li key={strList.id}> {strList} ;
});

```

## Example: Correct Key usage

To correct the above example, we should have to assign key to the map() iterator.

```
import React from 'react';
import ReactDOM from 'react-dom';

function ListItem(props) {
 const item = props.item;
 return (
 // No need to specify the key here.
 {item}
);
}

function NameList(props) {
 const myLists = props.myLists;
 const listItems = myLists.map((strLists) =>
 // The key should have been specified here.
 <ListItem key={myLists.toString()} item={strLists} />
);
 return (
 <div>
 <h2>Correct Key Usage Example</h2>
 {listItems}
 </div>
);
}

const myLists = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];
ReactDOM.render(
 <NameList myLists={myLists}>/>,
 document.getElementById('app')
);
export default App;
```

## Output



## React Forms

Forms are an integral part of any modern web application. It allows the users to interact with the application as well as gather information from the users. Forms can perform many tasks that depend on the nature of your business requirements and logic such as authentication of the user, adding user, searching, filtering etc. A form can contain text fields, buttons, checkbox, radio button, etc.

### Creating Form

React offers a stateful, reactive approach to build a form. The component rather than the DOM usually handles the React form. In React, the form is usually implemented by using controlled components.

There are mainly two types of form input in React.

1. Uncontrolled component
2. Controlled component

### Uncontrolled component

The uncontrolled input is similar to the traditional HTML form inputs. The DOM itself handles the form data. Here, the HTML elements maintain their own state that will be updated when the input value changes. To write an uncontrolled component, you need to use a ref to get form values from the DOM. In other words, there is no need to write an event handler for every state update. You can use a ref to access the input field value of the form from the DOM.

In this example, the code accepts a field username and company name in an uncontrolled component.

```
import React, { Component } from 'react';
class App extends React.Component {
 constructor(props) {
 super(props);
 this.updateSubmit = this.updateSubmit.bind(this);
 this.input = React.createRef();
 }
 updateSubmit(event) {
 alert('You have entered the UserName and CompanyName successfully.');
```

event.preventDefault();

```
 }
 render() {
 return (
 <form onSubmit={this.updateSubmit}>
 <h1>Uncontrolled Form Example</h1>
 <label>Name:
 <input type="text" ref={this.input} />
 </label>
 <label>
 CompanyName:
 <input type="text" ref={this.input} />
 </label>
 <input type="submit" value="Submit" />
 </form>
);
 }
}
export default App;
```



## Output

When you execute the above code, you will see the following screen.



Uncontrolled Form Example

Name:  CompanyName:

After filling the data in the field, you get the message that can be seen in the below screen.



Uncontrolled Form Example

Name:  CompanyName:

localhost:8080 says  
You have entered the UserName and CompanyName successfully.

## Controlled Component

In HTML, form elements typically maintain their own state and update it according to the user input. In the controlled component, the input form element is handled by the component rather than the DOM. Here, the mutable state is kept in the state property and will be updated only with `setState()` method.

Controlled components have functions that govern the data passing into them on every `onChange` event, rather than grabbing the data only once, e.g., when you click a submit button. This data is then saved to state and updated with `setState()` method. This makes component have better control over the form elements and data.

```
import React, { Component } from 'react';
class App extends React.Component {
 constructor(props) {
 super(props);
 this.state = {value: ''};
 this.handleChange = this.handleChange.bind(this);
 this.handleSubmit = this.handleSubmit.bind(this);
 }
 handleChange(event) {
 this.setState({value: event.target.value});
 }
 handleSubmit(event) {
 alert('You have submitted the input successfully: ' + this.state.value);
 }
}
```

```

 event.preventDefault();
 }
 render() {
 return (
 <form onSubmit={this.handleSubmit}>
 <h1>Controlled Form Example</h1>
 <label>
 Name:
 <input type="text" value={this.state.value} onChange={this.handleChange} />
 </label>
 <input type="submit" value="Submit" />
 </form>
);
 }
}
export default App;

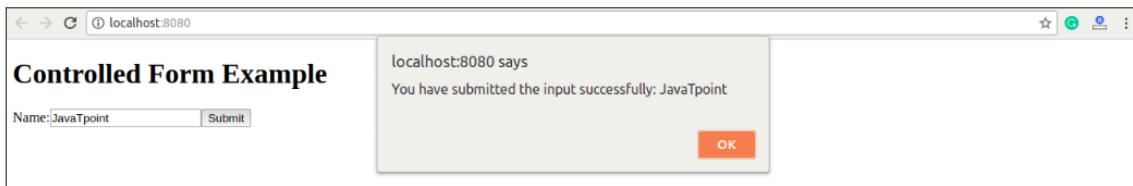
```

## Output

When you execute the above code, you will see the following screen.



After filling the data in the field, you get the message that can be seen in the below screen.



## Handling Multiple Inputs in Controlled Component

If you want to handle multiple controlled input elements, add a **name** attribute to each element, and then the handler function decided what to do based on the value of **event.target.name**.

# Unit-3

## Syllabus

**Unit III Routing and Component Communication: React Router (v6), Route Params, Nested Routes, Navigation, Passing Data via Props, Context API, useContext, Custom Hooks (Intro).**

### React Router

Routing is a process in which a user is directed to different pages based on their action or request. ReactJS Router is mainly used for developing Single Page Web Applications. React Router is used to define multiple routes in the application. When a user types a specific URL into the browser, and if this URL path matches any 'route' inside the router file, the user will be redirected to that particular route.

### Need of React Router

React Router plays an important role to display multiple views in a single page application. Without React Router, it is not possible to display multiple views in React applications. Most of the social media websites like Facebook, Instagram uses React Router for rendering multiple views

### React Router Installation

React contains three different packages for routing. These are:

1. react-router: It provides the core routing components and functions for the React Router applications.
2. react-router-native: It is used for mobile applications.
3. react-router-dom: It is used for web applications design.

## Components in React Router

There are two types of router components:

- `<BrowserRouter>`: It is used for handling the dynamic URL.
- `<HashRouter>`: It is used for handling the static request.

### Example

Step-1: In our project, we will create two more components along with App.js, which is already present.

About.js

```
import React from 'react'
class About extends React.Component {
 render() {
 return <h1>About</h1>
 }
}
export default About
```

Contact.js

```
import React from 'react'
class Contact extends React.Component {
 render() {
 return <h1>Contact</h1>
 }
}
export default Contact
```

App.js

```
import React from 'react'
class App extends React.Component {
 render() {
```

```

 return (
 <div>
 <h1>Home</h1>
 </div>
)
 }
}
export default App

```

Step-2: For Routing, open the index.js file and import all the three component files in it. Here, you need to import line: `import { Route, Link, BrowserRouter as Router } from 'react-router-dom'` which helps us to implement the Routing. Now, our index.js file looks like below.

## What is Route?

It is used to define and render component based on the specified path. It will accept components and render to define what should be rendered.

Index.js

```

import React from 'react';
import ReactDOM from 'react-dom';
import { Route, Link, BrowserRouter as Router } from 'react-router-dom'
import './index.css';
import App from './App';
import About from './about'
import Contact from './contact'

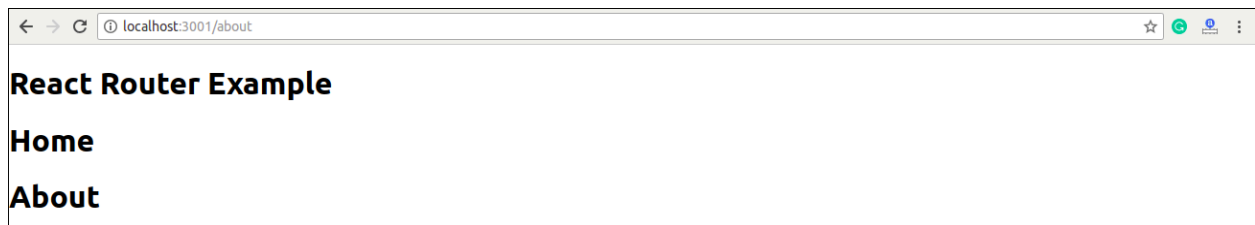
const routing = (
 <Router>
 <div>
 <h1>React Router Example</h1>
 <Route path="/" component={App} />
 <Route path="/about" component={About} />
 <Route path="/contact" component={Contact} />
 </div>
 </Router>
)
ReactDOM.render(routing, document.getElementById('root'));

```

Step-3: Open command prompt, go to your project location, and then type npm start. You will get the following screen.



Now, if you enter manually in the browser: localhost:3000/about, you will see About component is rendered on the screen.



Step-4: In the above screen, you can see that Home component is still rendered. It is because the home path is '/' and about path is '/about', so you can observe that slash is common in both paths which render both components. To stop this behavior, you need to use the exact prop. It can be seen in the below example.

Index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import { Route, Link, BrowserRouter as Router } from 'react-router-dom'
import './index.css';
import App from './App';
import About from './about'
import Contact from './contact'

const routing = (
 <Router>
 <div>
 <h1>React Router Example</h1>
 <Route exact path="/" component={App} />
 <Route path="/about" component={About} />
 <Route path="/contact" component={Contact} />
 </div>
 </Router>
```

```
)
ReactDOM.render(routing, document.getElementById('root'));
```

Output



## Adding Navigation using Link component

Sometimes, we want to need multiple links on a single page. When we click on any of that particular Link, it should load that page which is associated with that path without reloading the web page. To do this, we need to import `<Link>` component in the `index.js` file.

What is `<Link>` component?

This component is used to create links which allow to navigate on different URLs and render its content without reloading the webpage.

Example

Index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import { Route, Link, BrowserRouter as Router } from 'react-router-dom'
import './index.css';
import App from './App';
import About from './about'
import Contact from './contact'

const routing = (
 <Router>
 <div>
 <h1>React Router Example</h1>

 <Link to="/">Home</Link>

```

```


 <Link to="/about">About</Link>

 <Link to="/contact">Contact</Link>

 <Route exact path="/" component={App} />
 <Route path="/about" component={About} />
 <Route path="/contact" component={Contact} />
</div>
</Router>
)
ReactDOM.render(routing, document.getElementById('root'));

```

## Output



After adding Link, you can see that the routes are rendered on the screen. Now, if you click on the About, you will see URL is changing and About component is rendered.



Now, we need to add some styles to the Link. So that when we click on any particular link, it can be easily identified which Link is active. To do this react router provides a new trick NavLink instead of Link. Now, in the index.js file, replace Link from Navlink and add properties activeClassName. The activeClassName properties mean when we click on the Link, it should have a specific style so that we can differentiate which one is currently active.

```

import React from 'react';
import ReactDOM from 'react-dom';

```



```

import { BrowserRouter as Router, Route, Link, NavLink } from 'react-router-dom'
import './index.css';
import App from './App';
import About from './about'
import Contact from './contact'

const routing = (
 <Router>
 <div>
 <h1>React Router Example</h1>

 <NavLink to="/" exact activeStyle={
 {color:'red'}
 }>Home</NavLink>

 <NavLink to="/about" exact activeStyle={
 {color:'green'}
 }>About</NavLink>

 <NavLink to="/contact" exact activeStyle={
 {color:'magenta'}
 }>Contact</NavLink>

 <Route exact path="/" component={App} />
 <Route path="/about" component={About} />
 <Route path="/contact" component={Contact} />
 </div>
 </Router>
)
ReactDOM.render(routing, document.getElementById('root'));

```

## Output

When we execute the above program, we will get the following screen in which we can see that Home link is of color Red and is the only currently active link.



Now, when we click on About link, its color shown green that is the currently active link.



## <Link> vs <NavLink>

The Link component allows navigating the different routes on the websites, whereas NavLink component is used to add styles to the active routes.

## React Router Switch

The <Switch> component is used to render components only when the path will be matched. Otherwise, it returns to the not found component.

To understand this, first, we need to create a notfound component.

notfound.js

```
import React from 'react'
const Notfound = () => <h1>Not found</h1>
export default Notfound
```

Now, import component in the index.js file. It can be seen in the below code.

Index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import { BrowserRouter as Router, Route, Link, NavLink, Switch } from
'react-router-dom'
import './index.css';
import App from './App';
```

```
import About from './about'
import Contact from './contact'
import Notfound from './notfound'
```

```
const routing = (
 <Router>
 <div>
 <h1>React Router Example</h1>

 <NavLink to="/" exact activeStyle={
 {color:'red'}
 }>Home</NavLink>

 <NavLink to="/about" exact activeStyle={
 {color:'green'}
 }>About</NavLink>

 <NavLink to="/contact" exact activeStyle={
 {color:'magenta'}
 }>Contact</NavLink>

 <Switch>
 <Route exact path="/" component={App} />
 <Route path="/about" component={About} />
 <Route path="/contact" component={Contact} />
 <Route component={Notfound} />
 </Switch>
 </div>
 </Router>
)
ReactDOM.render(routing, document.getElementById('root'));
```

## Output

If we manually enter the wrong path, it will give the not found error.



## React Router <Redirect>

A <Redirect> component is used to redirect to another route in our application to maintain the old URLs. It can be placed anywhere in the route hierarchy.

# Stop

## Nested Routing in React

Nested routing allows you to render sub-routes in your application. It can be understood in the below example.

Example

index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import { BrowserRouter as Router, Route, Link, NavLink, Switch } from
'react-router-dom'
import './index.css';
import App from './App';
import About from './about'
import Contact from './contact'
import Notfound from './notfound'

const routing = (
 <Router>
 <div>
 <h1>React Router Example</h1>


```

```

 <NavLink to="/" exact activeStyle={
 {color:'red'}
 }>Home</NavLink>

 <NavLink to="/about" exact activeStyle={
 {color:'green'}
 }>About</NavLink>

 <NavLink to="/contact" exact activeStyle={
 {color:'magenta'}
 }>Contact</NavLink>

<Switch>
 <Route exact path="/" component={App} />
 <Route path="/about" component={About} />
 <Route path="/contact" component={Contact} />
 <Route component={NotFound} />
</Switch>
</div>
</Router>
)
ReactDOM.render(routing, document.getElementById('root'));

```

In the contact.js file, we need to import the React Router component to implement the subroutes.

contact.js

```

import React from 'react'
import { Route, Link } from 'react-router-dom'

const Contacts = ({ match }) => <p>{match.params.id}</p>

class Contact extends React.Component {
 render() {
 const { url } = this.props.match
 return (
 <div>

```

```

 <h1>Welcome to Contact Page</h1>
 Select contact Id

 <Link to="/contact/1">Contacts 1 </Link>

 <Link to="/contact/2">Contacts 2 </Link>

 <Link to="/contact/3">Contacts 3 </Link>

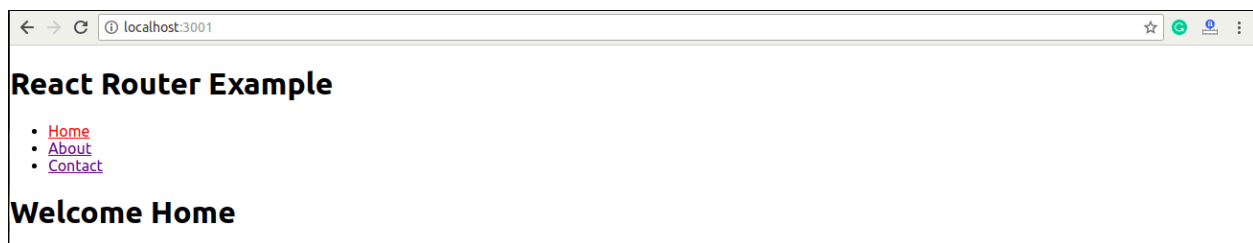
 <Link to="/contact/4">Contacts 4 </Link>

 <Route path="/contact/:id" component={Contacts} />
 </div>
)
}
}
export default Contact

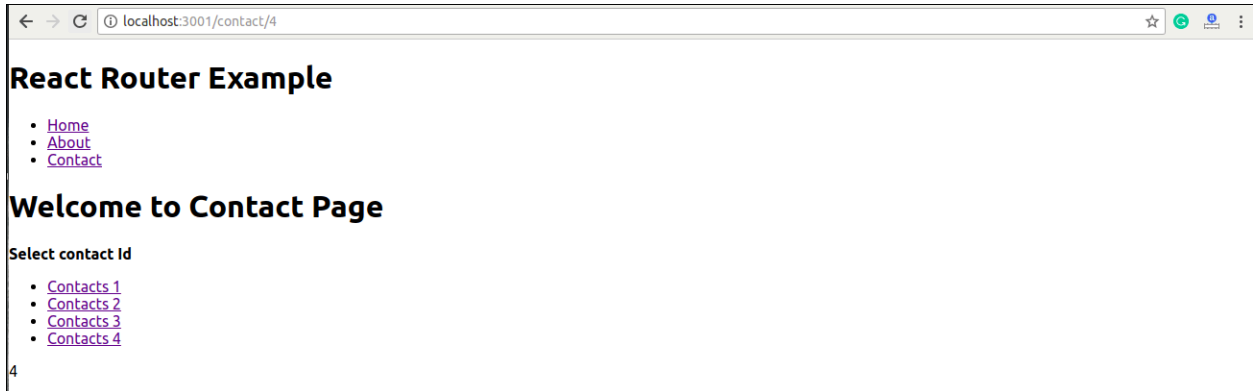
```

## Output

When we execute the above program, we will get the following output.



After clicking the Contact link, we will get the contact list. Now, selecting any contact, we will get the corresponding output. It can be shown in the below example.



## Benefits Of React Router

The benefits of React Router is given below:

- In this, it is not necessary to set the browser history manually.
- Link uses to navigate the internal links in the application. It is similar to the anchor tag.
- It uses Switch feature for rendering.
- The Router needs only a Single Child element.
- In this, every component is specified in.

## Data via Props in React

### What are Props?

- **Props** = short for **Properties**.
- Props are **read-only inputs** passed from a **parent component** → **child component**.
- They make components **reusable and dynamic**.
- Similar to **function parameters** in JavaScript.

👉 **Important:** Props are **immutable** (cannot be changed by the child component).

---

## ◆ 2. Passing Data via Props

### ✓ Example 1 – Simple Props

```
function Welcome(props) {
 return <h1>Hello, {props.name}!</h1>;
}
```

```
function App() {
 return (
 <div>
 <Welcome name="Krishna" />
 <Welcome name="Riya" />
 </div>
);
}
```

👉 Output:

Hello, Krishna!

Hello, Riya!

### Example 2 – Multiple Props

```
function Student(props) {
 return (
 <div>
 <h2>{props.name}</h2>
 <p>Roll No: {props.roll}</p>
 </div>
);
}
```

```
function App() {
 return (
 <div>
 <Student name="Ravi" roll="101" />
 <Student name="Priya" roll="102" />
 </div>
);
}
```



✓ Each **Student** component receives different **props values**.

---

### ✓ Example 3 – Destructuring Props

Instead of **props.name**, you can directly use **destructuring**:

```
function Student({ name, roll }) {
 return (
 <div>
 <h2>{name}</h2>
 <p>Roll No: {roll}</p>
 </div>
);
}
```

---

## ◆ 3. Default Props

If parent doesn't pass data, you can set **default props**.

```
function Greeting({ name = "Guest" }) {
 return <h2>Hello, {name}</h2>;
}
```

```
function App() {
 return (
 <div>
 <Greeting name="Krishna" />
 <Greeting /> {/* Will show "Guest" */}
 </div>
);
}
```

## 4. Props vs State

Feature	Props	State
Definition	Data passed from parent	Data managed inside component
Mutable?	✗ Immutable	✓ Mutable

Who updates?	Parent component	Component itself ( <code>useState</code> , hooks)
Usage	Make components reusable	Handle dynamic UI changes

---

## ◆ 5. Real-World Example – Expense Manager (Props in Action)

👉 Parent passes expense data **via props** to child component:

```
function ExpenseItem({ title, amount }) {
 return (
 <div>
 <h3>{title}</h3>
 <p>Amount: ₹{amount}</p>
 </div>
);
}

function App() {
 const expenses = [
 { title: "Book", amount: 300 },
 { title: "Pen", amount: 50 }
];

 return (
 <div>
 {expenses.map((exp, index) => (
 <ExpenseItem key={index} title={exp.title} amount={exp.amount} />
))}
 </div>
);
}
```

# 1. Context API in React

## 👉 What it is:

- Context API is used for **state sharing across multiple components** without passing props manually at every level (**props drilling problem**).

## ✅ Example: Using Context API

```
import React, { createContext } from "react";
```

```
// Step 1: Create Context
```

```
const UserContext = createContext();
```

```
function App() {
```

```
 const user = { name: "Krishna", role: "Admin" };
```

```
 return (
```

```
 // Step 2: Provide value to children
```

```
 <UserContext.Provider value={user}>
```

```
 <Dashboard />
```

```
 </UserContext.Provider>
```

```
);
```

```
}
```

```
function Dashboard() {
```

```
 return (
```

```
<div>

 <h2>Dashboard</h2>

 <Profile />

</div>

);
}

function Profile() {

 return (

 <UserContext.Consumer>

 {(user) => <h3>Hello {user.name}, Role: {user.role}</h3>}

 </UserContext.Consumer>

);
}

export default App;
```

✓ Here, **user** data is shared **without passing props** manually from **App** → **Dashboard** → **Profile**.

---

## 2. useContext Hook

👉 **What it is:**

- A React Hook that allows **direct access** to context values.
- Cleaner than `Context.Consumer`.

### ✓ Example: `useContext`

```
import React, { createContext, useContext } from "react";
```

```
// Create Context
```

```
const ThemeContext = createContext("light");
```

```
function App() {
```

```
 return (
```

```
 <ThemeContext.Provider value="dark">
```

```
 <Toolbar />
```

```
 </ThemeContext.Provider>
```

```
);
```

```
}
```

```
function Toolbar() {
```

```
 return (
```

```
 <div>
```

```
 <ThemeButton />
```

```
 </div>
```

```
);
```

```
}
```

```
function ThemeButton() {

 const theme = useContext(ThemeContext); // direct access

 return <button>Current Theme: {theme}</button>;

}
```

```
export default App;
```

✓ `useContext` makes consuming context **simpler and more readable**.

---

### 3. Custom Hooks (Intro)

👉 **What it is:**

- A **custom function** starting with `use` that lets you **reuse logic** between components.
- Helps keep components **clean**.

✓ **Example: Custom Hook – `useWindowSize`**

```
import React, { useState, useEffect } from "react";
```

```
// Step 1: Create custom hook
```

```
function useWindowSize() {

 const [size, setSize] = useState(window.innerWidth);

 useEffect(() => {
```

```
const handleResize = () => setSize(window.innerWidth);

window.addEventListener("resize", handleResize);

return () => window.removeEventListener("resize", handleResize);
}, []);

return size;
}

// Step 2: Use hook in components
function App() {
 const width = useWindowSize();

 return (
 <div>
 <h2>Window Width: {width}px</h2>
 </div>
);
}

export default App;
```

✓ `useWindowSize` can now be reused in **any component**.

---

## Summary

- **Context API** → Share state globally across components.
- **useContext Hook** → Simplifies consuming context values.
- **Custom Hooks** → Reusable logic for cleaner code.

# Unit IV

## Syllabus

API Integration and Real-World Features: Fetching data using fetch/axios, Asynchronous programming with Promises, API error handling, Loading states, Pagination, Search, Filters. LocalStorage, Custom Hooks (in detail).

## What is an API?

- **API (Application Programming Interface)** is a way for applications to **communicate** with each other.
- In React, we often use **REST APIs** to fetch data from a **server** and display it in the UI.
- Example: Weather API → provides temperature data for your app.

### Key Points about REST API:

- Uses **HTTP methods**:
  - **GET** → Fetch data
  - **POST** → Send new data



- **PUT/PATCH** → Update data
  - **DELETE** → Remove data
  - Returns **JSON** (JavaScript Object Notation), which is easy to use in React.
- 

## ◆ 2. Fetching Data in React

### (a) Using **fetch()** (native JavaScript API)

- Built into the browser.
- Returns a **Promise**.

#### Steps:

1. Call **fetch(url)**
2. Convert response into JSON → **.json()**
3. Handle success (**.then**) & error (**.catch**)

#### Example:

```
import React, { useState, useEffect } from "react";

function UsersFetch() {
 const [users, setUsers] = useState([]);

 useEffect(() => {
 fetch("https://jsonplaceholder.typicode.com/users")
 .then(response => response.json())
 .then(data => setUsers(data))
 .catch(error => console.log("Error:", error));
 }, []);
```

```

return (
 <div>
 <h2>Users List (Using Fetch)</h2>

 {users.map(user => (
 <li key={user.id}>{user.name} - {user.email}
))}

 </div>
);
}

export default UsersFetch;

```

---

## (b) Using **axios** (third-party library)

- Must be installed: `npm install axios`
- Automatically converts response into JSON.
- Easier syntax than `fetch()`.

### Example:

```

import React, { useState, useEffect } from "react";
import axios from "axios";

function UsersAxios() {
 const [users, setUsers] = useState([]);

 useEffect(() => {
 axios.get("https://jsonplaceholder.typicode.com/users")
 .then(response => setUsers(response.data))
 .catch(error => console.log("Error:", error));
 }, []);

 return (

```

```
 <div>
 <h2>Users List (Using Axios)</h2>

 {users.map(user => (
 <li key={user.id}>{user.name} - {user.email}
))}

 </div>
);
}

export default UsersAxios;
```

---

### ◆ 3. Asynchronous Programming in React

#### Promises

- **Promise** = Object that represents success (resolved) or failure (rejected) of an asynchronous operation.
- Example:

```
fetch("https://jsonplaceholder.typicode.com/posts")
 .then(res => res.json())
 .then(data => console.log(data))
 .catch(err => console.log(err));
```

#### async/await

- Cleaner, more readable than `.then()`.
- Must be used inside an `async` function.

#### Example:

```
useEffect(() => {
```

```

const fetchData = async () => {
 try {
 const res = await
fetch("https://jsonplaceholder.typicode.com/posts");
 const data = await res.json();
 console.log(data);
 } catch (error) {
 console.log(error);
 }
};
fetchData();
}, []);

```

API = Communication bridge between frontend & backend.

REST API returns JSON.

`fetch(url)` → returns Promise → `.then()` for data → `.catch()` for error.

`axios` is a library, simplifies requests.

`async/await` makes code cleaner than chaining `.then()`.

```

useEffect(() => {
 fetch("https://jsonplaceholder.typicode.com/users")
 .then(res => res.json())
 .then(data => setUsers(data))
 .catch(err => console.error(err));
}, []);

```

## Error Handling & Loading States

---

### ◆ 1. Why Handle Errors?

- API requests can fail due to:
  - Network issues
  - Wrong API endpoint
  - Server down
  - Invalid data format

If errors are not handled → app crashes or shows blank screen.

---

## ♦ 2. Handling API Errors

Using **.catch()** with fetch

```
fetch("https://wrong-url.com/data")
 .then(res => res.json())
 .catch(error => console.log("Error occurred:", error));
```

Using **try...catch** with async/await

```
try {
 const res = await
fetch("https://jsonplaceholder.typicode.com/wrongurl");
 if (!res.ok) throw new Error("Failed to fetch data");
 const data = await res.json();
} catch (error) {
 console.error("Error:", error.message);
}
```

---

## ♦ 3. Loading States

- **Problem:** Data fetching takes time → user sees empty page.

- **Solution:** Show a **loading spinner** or message until data arrives.

### Steps:

1. Create a state variable `loading`.
2. Set `loading=true` before API call.
3. Set `loading=false` after API success/error.
4. Conditionally render UI.

### Example:

```
import React, { useState, useEffect } from "react";

function Posts() {
 const [posts, setPosts] = useState([]);
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState("");

 useEffect(() => {
 const fetchPosts = async () => {
 try {
 const res = await
fetch("https://jsonplaceholder.typicode.com/posts");
 if (!res.ok) throw new Error("Network error!");
 const data = await res.json();
 setPosts(data);
 } catch (err) {
 setError(err.message);
 } finally {
 setLoading(false);
 }
 };
 fetchPosts();
 }, []);

 if (loading) return <p>Loading...</p>;
```

```
if (error) return <p style={{color:"red"}}>{error}</p>;

return (

 {posts.slice(0, 5).map(post => (
 <li key={post.id}>{post.title}
))}

);
}

export default Posts;
```

---

## ◆ 4. Best Practices

- Always **show a loader** before data is ready.
  - Always **catch errors** and show user-friendly messages.
  - Use **finally {}** in async/await to reset **loading**.
  - Don't block UI; let users know something is happening.
- 

Students should know how to:

1. Catch API errors (**catch**, **try/catch**).
  2. Show loading messages or spinners.
  3. Provide fallback UI when request fails.
- 

📌 **Summary of Day 1 & 2 (Together):**

- API basics (REST, JSON, HTTP methods).
- Fetching data (`fetch`, `axios`).
- Promises & `async/await`.
- Error handling.
- Loading states.

## Error Handling & Loading States

### Topics:

- Handling API errors (`try/catch`, `.catch()`).
- Loading states (`isLoading`, skeleton UI).
- Displaying error messages.

### Notes (Theory):

- Always show feedback → "Loading..." before data arrives.
- Handle network failures (wrong URL, server down).
- Use `try...catch` in `async/await`.

### Code Snippet:

```
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
 const fetchData = async () => {
 try {
 const res = await
fetch("https://jsonplaceholder.typicode.com/posts");
 if (!res.ok) throw new Error("Network Error");
```



```
 const data = await res.json();
 setData(data);
 } catch (err) {
 setError(err.message);
 } finally {
 setLoading(false);
 }
};
fetchData();
}, []);
```

## Pagination

### Why?

- APIs often return **large datasets** (1000+ records).
- Pagination means **splitting data into pages** (e.g., 10 results per page).
- Improves **performance & user experience**.

### Types:

#### 1. Client-side Pagination:

- Load all data once, then split into pages.
- Good for **small datasets**.

#### 2. Server-side Pagination:

- Fetch data **page by page** using API parameters like `?page=2&limit=5`.
- Good for **large datasets**.

### Example (Client-side Pagination):

```
import React, { useState, useEffect } from "react";
```

```
function PaginationExample() {
 const [posts, setPosts] = useState([]);
 const [page, setPage] = useState(1);
 const postsPerPage = 5;

 useEffect(() => {
 fetch("https://jsonplaceholder.typicode.com/posts")
 .then(res => res.json())
 .then(data => setPosts(data));
 }, []);

 const start = (page - 1) * postsPerPage;
 const paginatedPosts = posts.slice(start, start + postsPerPage);

 return (
 <div>
 <h2>Paginated Posts</h2>

 {paginatedPosts.map(post => (
 <li key={post.id}>{post.title}
))}

 <button onClick={() => setPage(page - 1)} disabled={page === 1}>
 Prev
 </button>
 <button onClick={() => setPage(page + 1)} disabled={start +
postsPerPage >= posts.length}>
 Next
 </button>
 </div>
);
}

export default PaginationExample;
```

---

## ◆ 2. Search

### Why?

- Users want to **find specific data** quickly.
- Example: Search students by name, filter products by title.

### Example:

```
const [search, setSearch] = useState("");

const filteredPosts = posts.filter(post =>
 post.title.toLowerCase().includes(search.toLowerCase())
);

<input
 type="text"
 placeholder="Search posts..."
 value={search}
 onChange={(e) => setSearch(e.target.value)}
/>

 {filteredPosts.map(post => <li key={post.id}>{post.title})}

```

---

## ◆ 3. Filters

### Why?

- Filtering narrows down results based on **criteria** (category, price, rating).

### Example:

```
const [filterUserId, setFilterUserId] = useState(1);
```

```
const filteredPosts = posts.filter(post => post.userId ===
filterUserId);

<select onChange={(e) => setFilterUserId(Number(e.target.value))}>
 <option value="1">User 1</option>
 <option value="2">User 2</option>
</select>

 {filteredPosts.map(post => <li key={post.id}>{post.title})}

```

---

✓ **Outcome:**

- Understand Pagination, Search, Filters.
  - Build interactive apps that handle **large datasets** efficiently.
- 

## – LocalStorage

---

### ◆ 1. What is LocalStorage?

- Browser-based **key-value storage**.
- Stores data **permanently** (until cleared manually).
- Capacity: ~5MB per domain.

#### Use Cases:

- Save **user preferences** (theme, language).

- Cache **API data** (avoid repeated requests).
  - Store **authentication tokens**.
- 

## ◆ 2. LocalStorage Methods

- `localStorage.setItem("key", "value")` → store
  - `localStorage.getItem("key")` → retrieve
  - `localStorage.removeItem("key")` → delete one
  - `localStorage.clear()` → delete all
- 

## ◆ 3. Example: Save API Data in LocalStorage

```
import React, { useState, useEffect } from "react";

function LocalStorageExample() {
 const [users, setUsers] = useState([]);

 useEffect(() => {
 const cached = localStorage.getItem("users");
 if (cached) {
 setUsers(JSON.parse(cached));
 } else {
 fetch("https://jsonplaceholder.typicode.com/users")
 .then(res => res.json())
 .then(data => {
 setUsers(data);
 localStorage.setItem("users", JSON.stringify(data));
 });
 }
 }, []);
}
```



- Always start with **use** (e.g., `useFetch`, `useAuth`).
  - Must be used **inside a component or another hook**.
  - Cannot be used in loops or conditions.
- 

### ◆ 3. Example: Custom Hook for Fetching Data

#### `useFetch.js`

```
import { useState, useEffect } from "react";

function useFetch(url) {
 const [data, setData] = useState([]);
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState(null);

 useEffect(() => {
 const fetchData = async () => {
 try {
 const res = await fetch(url);
 if (!res.ok) throw new Error("Error fetching data");
 const json = await res.json();
 setData(json);
 } catch (err) {
 setError(err.message);
 } finally {
 setLoading(false);
 }
 };
 fetchData();
 }, [url]);

 return { data, loading, error };
}

export default useFetch;
```

---

### Posts.js (Using the hook)

```
import useFetch from "../useFetch";

function Posts() {
 const { data: posts, loading, error } =
 useFetch("https://jsonplaceholder.typicode.com/posts");

 if (loading) return <p>Loading...</p>;
 if (error) return <p style={{color:"red"}}>{error}</p>;

 return (

 {posts.slice(0, 5).map(p => <li key={p.id}>{p.title})}

);
}

export default Posts;
```

---

## ◆ 4. Benefits of Custom Hooks

- **Reusability** → Write once, use everywhere.
- **Separation of Concerns** → Keep UI code separate from logic.
- **Cleaner Components** → Easier to read & maintain.

---

### ✓ Outcome:

- Students can **write custom hooks** for APIs, authentication, forms.
- Can build **clean, scalable React apps**.
-



- 
- 
- 

# Unit -5

Project Development and Deployment: Building a complete project (e.g., Blog, E-commerce UI), Component structure, Styling in React (CSS Modules / Styled Components), Environment Variables, Deployment on Netlify/Vercel, Code splitting, Performance optimization

## 1. Building a Complete Project

When building a **React project** (Blog, E-commerce UI, Dashboard, etc.), we follow these steps:

1. **Plan Project** – Define features (login, product listing, blog posts).
  2. **Component Structure** – Break UI into reusable parts.
    - Example (E-commerce):
      - `Header`, `Footer`, `ProductCard`, `Cart`, `Checkout`.
    - Example (Blog):
      - `Navbar`, `PostList`, `SinglePost`, `Comments`.
  3. **State Management** – Use `useState`, `useReducer`, or Context API.
  4. **API Integration** – Fetch products/posts from API.
  5. **Styling** – Apply CSS Modules or Styled Components.
  6. **Testing & Debugging**.
  7. **Build & Deploy** (Netlify / Vercel).
-

## ◆ 2. Component Structure

- **Reusable Components** → avoid repetition.
- **Folder Organization Example:**

```
src/
├── components/
│ ├── Header.js
│ ├── Footer.js
│ ├── ProductCard.js
│ └── Post.js
├── pages/
│ ├── Home.js
│ ├── Blog.js
│ └── About.js
├── hooks/
│ └── useFetch.js
├── App.js
└── index.js
```

---

## ◆ 3. Styling in React

### (a) CSS Modules

- File naming: `Component.module.css`.
- Automatically scopes CSS → avoids conflicts.

#### Example:

```
import styles from "./Button.module.css";

function Button() {
 return <button className={styles.primary}>Click Me</button>;
}
export default Button;
```

```
/* Button.module.css */
.primary {
 background-color: blue;
 color: white;
}
```

---

## (b) Styled Components

- Install: `npm install styled-components`.
- CSS inside JS → dynamic styling.

### Example:

```
import styled from "styled-components";

const Button = styled.button`
 background: green;
 color: white;
 padding: 10px;
`;

export default function App() {
 return <Button>Click Me</Button>;
}
```

---

## ◆ 4. Environment Variables

- Used for **API keys** (e.g., weather API, Firebase).
- Create `.env` file at project root.

```
REACT_APP_API_KEY=yourapikey123
```

### Usage in React:

```
const apiKey = process.env.REACT_APP_API_KEY;
```

⚠ Always prefix with `REACT_APP_` in React.

---

## ◆ 5. Deployment (Netlify & Vercel)

### (a) Netlify

Build React App:

```
npm run build
```

- 1.
2. Drag & drop `build/` folder to Netlify.
3. Or connect GitHub repo → Auto deploy on push.

### (b) Vercel

Install CLI:

```
npm i -g vercel
```

- 1.

Run:

```
vercel
```

- 2.
3. Or connect GitHub repo → deploy automatically.

✓ Both provide **free SSL, CDN, and auto-deployment**.

---

## ◆ 6. Code Splitting

- **Problem:** Large apps → big JS file → slow load.
- **Solution:** Split code into smaller chunks.

React uses **dynamic import**:

```
import React, { Suspense, lazy } from "react";

const Blog = lazy(() => import("./Blog"));

function App() {
 return (
 <Suspense fallback=<h2>Loading...</h2>>
 <Blog />
 </Suspense>
);
}
```

✅ Loads components **only when needed** (improves performance).

---

## ◆ 7. Performance Optimization

**Techniques:**

1. **Memoization:**

- Use `React.memo()` to prevent unnecessary re-renders.

```
const Button = React.memo(({ text }) => <button>{text}</button>);
```

2.

3. **useCallback & useMemo:**

- Prevents recreation of functions/values.

4. **Virtualization (react-window):**

- For huge lists, render only visible items.

#### 5. **Image Optimization:**

- Use compressed images.
- Lazy-load images (`loading="lazy"`).

#### 6. **Bundle Optimization:**

- Tree-shaking (remove unused imports).
  - Dynamic imports for large libraries.
- 

## ◆ 8. Mini Project Ideas

### 1. **Blog App**

- Features: Create posts, list posts, search posts.
- Uses: Components, LocalStorage, API fetch, custom hook.

### 2. **E-commerce UI**

- Features: Product listing, search, filter, cart.
- Uses: Context API, styling modules, pagination.

### 3. **Weather App**

- Features: Search city, show weather via API.
- Uses: API integration, environment variables.

